What I Learned: Robotic Arms Track

By Ekan | Aurora Educators Program (AEP), Aurora Robotics

Over five weeks of the Aurora Educators Program, I went from blinking an LED to building a robotic arm that moves by hand or on its own.

What I learned

- **How a robot works.** A robot senses, decides, and acts. The Arduino is the brain, sensors are the inputs, and servos are the outputs.
- **Programming basics.** Every sketch has `setup()` (runs once) and `loop()` (runs forever). I used variables, `if / else`, `digitalRead` and `digitalWrite`, and the Serial Monitor to debug.
- **Servo control.** A servo is closed loop: it measures its own angle and corrects itself. The pulse width of a 50 Hz PWM signal sets the angle. To follow a knob, I read it, map it to 0 to 180 degrees, and write it to the servo.
- **Distance sensing.** The HC-SR04 times an echo. Distance in cm = echo time in microseconds x 0.034 / 2. A reading of 0 means no echo, so a robot must check for it.
- **Wireless control.** Bluetooth (HC-05) carries serial data, and a command protocol gives each character a meaning, like '1' for on or 'S1:90' for joint 1 to 90 degrees.
- **Safe power.** Servos draw too much current for the Arduino, so real builds need a separate 5 to 6 V supply with a shared ground.
- **Workflow.** Simulate in Tinkercad first, test one thing at a time, and commit the work to GitHub.

How it came together in the capstone

The capstone combined these into one Tinkercad arm using only a microcontroller, an ultrasonic sensor, a buzzer, a potentiometer, and servos. A `mode` variable decides whether the potentiometer drives a joint (manual) or the ultrasonic sensor drives a pick-and-place routine (autonomous).

Two problems taught me the most:

- **No spare button.** I turned the ultrasonic sensor into a mode switch: hold an object under 8 cm for one second to change mode, and the buzzer confirms each change.
- **Endless re-picking.** The arm kept picking the same object because it stayed in range. The Serial Monitor's distance readings showed why, and I fixed it by making the arm wait until the object was cleared.

Next steps

Replace `delay()` with `millis()` timing, add Bluetooth control, and scale up to a full 6-DOF arm with a PCA9685 driver.